

Curso

Introducción a los servicios de AWS

Módulo 1:

Fundamentos de la nube y servicios principales de AWS

Unidad 3:

Servicios de Cómputo: EC2, Containers y Serverless



Presentación

En esta unidad profundizaremos en una serie de servicios clave de Amazon Web Services (AWS) que permiten construir, desplegar y administrar aplicaciones y cargas de trabajo en la nube con alta disponibilidad, escalabilidad y flexibilidad. Estos servicios abarcan desde infraestructura básica, como instancias EC2, hasta plataformas de orquestación de contenedores y entornos serverless, cubriendo un amplio espectro de escenarios tecnológicos.

Comenzaremos analizando Amazon EC2 (Elastic Compute Cloud), el servicio fundamental de infraestructura como servicio (IaaS) que permite crear y ejecutar servidores virtuales en la nube con distintos sistemas operativos, tamaños y configuraciones. Se explorará cómo las instancias EC2 permiten replicar entornos locales, hospedar aplicaciones críticas y servir como base para arquitecturas más complejas. También se estudiará la importancia de Auto Scaling, que facilita el escalado automático de instancias según la demanda, y de Elastic Load Balancing (ELB), que distribuye el tráfico entre instancias saludables para garantizar alta disponibilidad y un rendimiento consistente.

Luego nos adentraremos en el mundo de los contenedores en AWS, revisando Amazon ECS (Elastic Container Service) como una solución gestionada para ejecutar y escalar aplicaciones en contenedores de manera eficiente. ECS permite definir clústeres, servicios y tareas que facilitan la orquestación de contenedores, mientras que AWS Fargate proporciona una capa serverless que

elimina la necesidad de gestionar servidores o clústeres subyacentes, ofreciendo despliegue instantáneo y pago por consumo. Estas herramientas son ideales para entornos de microservicios, cargas puntuales o aplicaciones que requieren flexibilidad y portabilidad.

Finalmente, abordaremos el paradigma serverless con AWS Lambda, un servicio que permite ejecutar código en respuesta a eventos sin necesidad de administrar servidores. Lambda habilita un modelo de pago por uso y escalado automático, y puede integrarse con otros servicios de AWS como S3, DynamoDB, API Gateway o EventBridge para construir arquitecturas orientadas a eventos, optimizar costos y simplificar operaciones. Se estudiarán los casos de uso más comunes, como procesamiento de datos en tiempo real, automatización de flujos de trabajo y creación de APIs ligeras.

Esta unidad proporciona los fundamentos para comprender cómo combinar instancias EC2, contenedores y funciones serverless, permitiendo diseñar soluciones escalables, eficientes y adaptadas a distintos escenarios en la nube de AWS.



Objetivos

Que los participantes logren...

- Comprender el funcionamiento de Amazon EC2 y su papel como servicio fundamental de infraestructura en la nube.
- Analizar cómo Auto Scaling y Elastic Load Balancing (ELB) permiten lograr escalabilidad, alta disponibilidad y distribución eficiente del tráfico en AWS.
- Identificar las características y casos de uso de Amazon ECS y AWS Fargate, así como sus diferencias y posibles integraciones en arquitecturas de contenedores.
- Explorar el concepto de arquitectura serverless y las capacidades de AWS Lambda para ejecutar código en respuesta a eventos sin gestión de infraestructura.



Bloques temáticos

1. Amazon EC2 (Elastic Compute Cloud)
 - 1.1 ¿Para qué se usa EC2?
 - 1.2 Componentes principales de una instancia EC2
 - 1.3 Ventajas de usar Amazon EC2
 - 1.4 Tamaños de máquinas virtuales
 - 1.5 Factores a considerar al elegir el tamaño de una instancia EC2
 - 1.6 Sistemas Operativos en EC2
 - 1.7 Componentes de Red en EC2
2. ¿Cómo lograr escalabilidad?
 - 2.1 AWS Auto Scaling
 - 2.2 Elastic Load Balancing (ELB)
 - 2.3 Modos de escalado: vertical vs horizontal
3. Contenedores en AWS
 - 3.1 Amazon ECS (Elastic Container Service)
 - 3.2 AWS Fargate
 - 3.3 ECS vs Fargate: Comparación práctica
4. Código en la nube: Introducción a AWS Lambda (serverless)
5. Conclusión
6. Glosario de conceptos clave

1. Amazon EC2 (Elastic Compute Cloud)

Una instancia de **Amazon EC2** es una máquina virtual segura y escalable en la nube de AWS. Funciona como una computadora dentro de otra, con su propio sistema operativo, CPU, memoria, almacenamiento y red, pero ejecutada sobre infraestructura de Amazon. Con EC2, los usuarios pueden lanzar servidores virtuales en minutos, escalar su capacidad de cómputo según la demanda y pagar únicamente por el tiempo y los recursos utilizados.

EC2 es uno de los servicios fundamentales de AWS dentro del modelo Infraestructura como Servicio (IaaS), y permite desplegar aplicaciones, bases de datos, entornos de prueba, servidores web y prácticamente cualquier carga de trabajo en la nube.

En esta sección vamos a comprender cómo funciona, sus componentes principales y las ventajas de utilizar EC2. También revisaremos los diferentes tipos de instancias, los sistemas operativos disponibles y los factores clave para seleccionar la configuración adecuada en función de las necesidades de cada caso.

1.1. ¿Para qué se usa EC2?

Las instancias de EC2 son extremadamente versátiles. Algunos de los usos más comunes incluyen:

- **Hospedaje de aplicaciones empresariales** que requieren control total del sistema operativo.

- **Servidores de desarrollo y pruebas**, replicando entornos de producción sin necesidad de comprar hardware.
- **Migración de cargas de trabajo on-premise** hacia la nube.
- **Ejecución de aplicaciones legacy** que aún no pueden migrar a arquitecturas serverless o de contenedores.
- **Computación de alto rendimiento (HPC)** para análisis de datos, procesamiento por lotes o simulaciones científicas.
- **Servidores web y aplicaciones globales**, con escalabilidad automática en función de la demanda.

1.2 Componentes principales de una instancia EC2

- **Instancia EC2:** es el servidor virtual en la nube que proporciona la capacidad de cómputo. Cada instancia se crea a partir de una Amazon Machine Image (AMI), que define el sistema operativo y el software base. Una vez lanzada, la instancia funciona como un equipo físico tradicional, pero con la flexibilidad de la nube para escalar, detenerse o reiniciarse según sea necesario.
- **Amazon Machine Image (AMI):** La AMI es la plantilla que sirve como punto de partida para crear una instancia. Define qué sistema operativo tendrá la máquina (Windows, Ubuntu, Red Hat, entre otros) y qué configuraciones iniciales estarán disponibles. AWS ofrece AMIs estándar, pero también se pueden obtener a través del Marketplace o incluso crear imágenes personalizadas, adaptadas a las necesidades específicas de cada organización.

- **Elastic Block Store (EBS):** El almacenamiento asociado a una instancia EC2 se gestiona a través de EBS, que proporciona volúmenes de disco virtual. Estos volúmenes pueden ser de dos tipos principales: el disco del sistema operativo, que contiene el SO y archivos base, y discos adicionales que se utilizan para aplicaciones, bases de datos o almacenamiento persistente. Una ventaja clave de EBS es que los volúmenes son persistentes, lo que significa que los datos se conservan incluso si la instancia se detiene.
- **CPU y memoria:** Los recursos de cómputo de una instancia están definidos por el tipo de instancia seleccionado. AWS ofrece una gran variedad de familias y tamaños que combinan distintos niveles de CPU, memoria y almacenamiento. Por ejemplo, instancias de uso general (**t3.micro**), optimizadas para memoria (**r6i.large**) o para cómputo intensivo (**c7g.xlarge**). La elección del tipo adecuado depende del tipo de carga de trabajo que se planea ejecutar.
- **Elastic Network Interface (ENI):** Cada instancia EC2 necesita conectividad, lo cual se logra a través de una interfaz de red elástica (ENI). Este componente permite que la instancia se conecte a una VPC (Virtual Private Cloud) y cuente con direcciones IP privadas para comunicación interna, además de una IP pública opcional para el acceso externo. En escenarios más avanzados, una instancia puede tener múltiples ENIs para gestionar diferentes flujos de tráfico.
- **Security Groups:** Los grupos de seguridad funcionan como firewalls virtuales que controlan el tráfico hacia y desde la instancia. Cada grupo de seguridad se configura con reglas específicas que permiten o deniegan conexiones según protocolos (TCP, UDP), puertos (por ejemplo, 22 para SSH o 3389 para RDP) y direcciones de origen o destino. Gracias a este mecanismo, la seguridad de red en EC2 puede gestionarse de forma granular y flexible.

1.3 Ventajas de usar Amazon EC2

Características de AWS EC2



- **Escalabilidad bajo demanda:** lanzar, detener o redimensionar instancias en minutos.
- **Pago por uso:** se paga solo por lo que se consume (on-demand, por segundo en la mayoría de los casos).
- **Diversidad de instancias:** optimizadas para CPU, memoria, almacenamiento o GPU.
- **Alta disponibilidad:** posibilidad de desplegar instancias en múltiples zonas de disponibilidad (AZs).
- **Integración con otros servicios AWS:** como S3, RDS, ELB, IAM, entre otros.

Si lo desean, pueden explorar la calculadora oficial de AWS para estimar costos de instancias EC2: <https://calculator.aws/#/createCalculator/ec2-enhancement>.

1.4 Tamaños de máquinas virtuales

Azure ofrece una gran variedad de tamaños de máquinas virtuales, según el tipo de carga de trabajo:

Series generales (uso balanceado)	Ejemplo: t3, t4g, m6i. Ofrecen un balance entre CPU, memoria y almacenamiento.
Series optimizadas para computo	Ejemplo: c6i, c7g. Ofrecen alto rendimiento de CPU, ideal para procesamiento intensivo.
Series optimizadas para memoria	Ejemplo: r6i, c7g. Poseen gran cantidad de RAM, útil para bases de datos en memoria.
Series optimizadas para almacenamiento	Ejemplo: i3, i4i, d2. Diseñadas para cargas con I/O intensivo como NoSQL o data lakes.
Series optimizadas para GPU	p4d, g5, inf1. Ideales para Machine Learning, renderizado o simulaciones.

1.5 Factores a considerar al elegir un tipo de instancia

Existen muchísimos factores al momento de decidir cuál es el tipo de instancia EC2 que mejor se adapte a nuestras necesidades, pero los más importantes son:

- **Tipo de carga de trabajo:** si se requiere CPU intensiva, memoria alta o GPU.

- **Escalabilidad:** número de usuarios o procesos concurrentes.
- **Disponibilidad regional:** no todos los tipos de instancias están en todas las regiones.
- **Costo:** considerar presupuesto y modelo de precios (on-demand, spot, reserved, savings plans).

1.6 Sistemas Operativos en EC2

Uno de los elementos clave a la hora de crear una instancia EC2 es la selección del sistema operativo (SO). AWS permite desplegar VMs tanto con **sistemas operativos Windows** como con **distribuciones Linux**, lo que lo convierte en una plataforma altamente versátil para cargas de trabajo diversas.

Windows	Linux
● Utilizado comúnmente en entornos corporativos. ● Ideal para ejecutar software como SQL Server, Active Directory o aplicaciones .NET. ● Requiere licenciamiento, aunque AWS lo incluye dentro del costo de la VM.	● Popular para desarrollo, testing, servidores web y aplicaciones de código abierto. ● Distribuciones soportadas: Ubuntu, Amazon Linux, Red Hat Enterprise Linux (RHEL), SUSE, Debian, CentOS, entre otras. ● Generalmente más ligero y con costos operativos menores que Windows.

1.7 Componentes de Red en EC2

Cuando se lanza una instancia, se configura automáticamente dentro de una **VPC (Virtual Private Cloud)**, con varios elementos clave:

1. **ENI (Elastic Network Interface):** Define la IP privada y reglas de conectividad. Puede asociarse a múltiples grupos de seguridad.
2. **Elastic IP:** Dirección IP pública estática que se puede asignar a la instancia. Muy útil para servidores que necesitan una IP fija accesible desde Internet.
3. **Security Groups:** Firewalls virtuales a nivel de instancia. Controlan qué tráfico entra o sale (ej: habilitar SSH en el puerto 22 o RDP en el 3389).

En resumen, una instancia **EC2** no es solo una VM aislada, sino parte de un ecosistema más amplio que involucra red, almacenamiento, seguridad e integración con el resto de servicios AWS.

2. ¿Cómo lograr escalabilidad?

La **escalabilidad** es uno de los pilares fundamentales de la computación en la nube. En AWS, se refiere a la capacidad de aumentar o disminuir recursos de forma automática o manual según la demanda de la carga de trabajo. Este principio permite que las aplicaciones mantengan un rendimiento constante y un uso eficiente de los recursos sin necesidad de intervención manual.

En el ecosistema de AWS, la escalabilidad se logra principalmente mediante dos servicios clave:

- **Auto Scaling Groups (ASG)**
- **Elastic Load Balancing (ELB)**

Estos servicios permiten que una aplicación se adapte al tráfico cambiante, garantizando disponibilidad, resiliencia y optimización de costos.

2.1. AWS Auto Scaling

Un **Auto Scaling Group (ASG)** es un conjunto de instancias EC2 que se gestionan de manera conjunta para responder dinámicamente a la demanda. El objetivo principal es mantener el número adecuado de instancias en funcionamiento, agregando nuevas cuando la carga aumenta y eliminando las innecesarias cuando la demanda baja.

Entre sus características principales se destacan:

- **Escalado automático:** basado en reglas predefinidas (por ejemplo, cuando el uso de CPU supera el 70% o según métricas personalizadas de CloudWatch).
- **Elasticidad:** permite manejar picos de tráfico sin necesidad de intervención manual.
- **Integración con ELB:** se combina con balanceadores de carga para asegurar que las nuevas instancias reciban tráfico inmediatamente al crearse.
- **Alta disponibilidad:** al distribuir las instancias en múltiples Zonas de Disponibilidad (AZs), se garantiza tolerancia a fallos.

En resumen, Auto Scaling es la herramienta clave para asegurar que una aplicación en AWS se mantenga ágil y resiliente frente a variaciones en la demanda.

2.2 Elastic Load Balancing (ELB)

El servicio **Elastic Load Balancing** distribuye automáticamente el tráfico entrante entre múltiples instancias EC2, contenedores o funciones Lambda. De esta forma, ninguna instancia individual se sobrecarga y los usuarios disfrutan de tiempos de respuesta consistentes.

Existen tres tipos principales de balanceadores en AWS:

- **Application Load Balancer (ALB)**: optimizado para aplicaciones web, funciona a nivel de capa 7 (HTTP/HTTPS) y permite balanceo avanzado con reglas basadas en rutas o cabeceras.
- **Network Load Balancer (NLB)**: diseñado para cargas de alto rendimiento y baja latencia, opera a nivel de capa 4 (TCP/UDP).
- **Gateway Load Balancer (GWLB)**: permite distribuir tráfico hacia dispositivos virtuales de terceros, como firewalls o sistemas de inspección de red.

Una de las funcionalidades clave del ELB es el uso de **health checks** o sondas de estado, que verifican continuamente la salud de las instancias. Si alguna presenta problemas, el balanceador deja de enviarle tráfico hasta que vuelva a estar operativa. Además, al integrarse con múltiples zonas de disponibilidad, los balanceadores de carga de AWS proporcionan alta disponibilidad y resiliencia ante fallas en centros de datos específicos.

2.3 Modos de escalado: vertical vs horizontal

En AWS, los modos de escalado principales son el **escalado vertical** y el **escalado horizontal**. El escalado vertical implica aumentar los recursos de una máquina virtual (como CPU, RAM) o aumentar la capacidad de una base de datos. El escalado horizontal implica agregar más instancias de recursos, como máquinas virtuales adicionales o bases de datos, para distribuir la carga

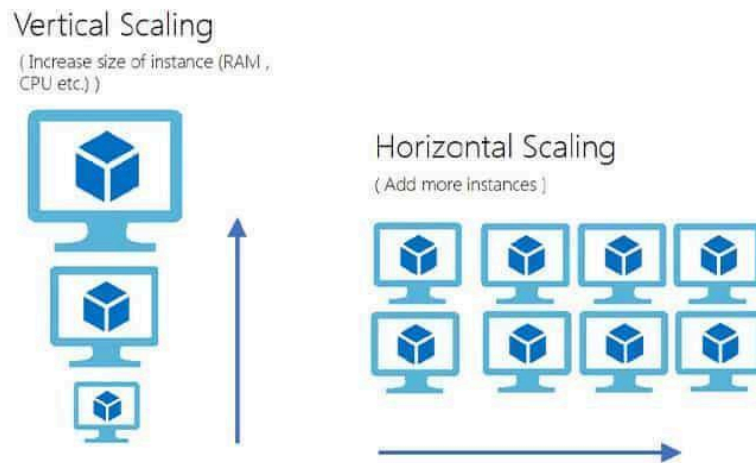


Figura 2: Diferencia entre escalado vertical y horizontal.

En el **escalado vertical** (Vertical Scaling / Scale Up) la idea principal consiste en aumentar la capacidad de un único servidor o instancia para que pueda manejar más carga. Esto puede lograrse agregando más CPU (núcleos más rápidos o más cantidad) o memoria RAM, mejorando el almacenamiento (más rápido o con mayor capacidad), reemplazando discos HDD por SSD, etc. Dentro de las ventajas, destaca la simplicidad ya que no se modifica la arquitectura, solamente se actualiza el hardware o el tamaño de la máquina.

En el **escalado horizontal** (Horizontal Scaling / Scale Out) la idea principal consiste en añadir más servidores o instancias que trabajen en paralelo para distribuir la carga. Esto puede lograrse colocando múltiples máquinas detrás de un balanceador de carga, también replicando bases de datos y particionandolas (sharding/particionamiento), etc. Dentro de las ventajas, destaca la escalabilidad casi ilimitada ya que la misma es posible siempre y cuando puedas añadir más nodos (por ende, mayor costo). También la alta disponibilidad ya que si un nodo falla, otros pueden seguir funcionando.

Aspecto	Escalado Vertical	Escalado Horizontal
Idea	Aumentar recursos de un servidor	Añadir más servidores en paralelo
Cómo	Más CPU/RAM/almacenamiento en la misma máquina	Balanceador de carga + replicación/particionamiento
Arquitectura	No cambia	Cambia, requiere coordinación
Ventaja	Simplicidad	Escalabilidad y alta disponibilidad
Limitación	Límite físico y costo	Complejidad y costo por nodos
Ejemplo	Aumentar tamaño de una VM	Varias VMs detrás de un Load Balancer

3. Contenedores en AWS

Los **contenedores** son unidades ligeras que empaquetan una aplicación junto con sus dependencias (librerías, runtime, binarios) para que se ejecute de forma consistente en cualquier entorno.

En la nube, los contenedores permiten desplegar aplicaciones de forma ágil, reproducible y con alto aprovechamiento de recursos.

En AWS, las principales opciones para ejecutar contenedores son **Amazon ECS (Elastic Container Service)** y **AWS Fargate**. ECS es un servicio de orquestación de contenedores totalmente gestionado, mientras que Fargate es un motor *serverless* para ejecutar contenedores sin necesidad de administrar servidores o clusters.

3.1 Amazon ECS (Elastic Container Service)

Amazon ECS es un servicio de orquestación que permite ejecutar, detener y administrar contenedores en un clúster de instancias EC2 o usando Fargate. Con ECS, los usuarios definen **tareas** (tasks) que describen cómo se deben ejecutar los contenedores: qué imagen utilizar, cuánta CPU y memoria asignar, qué variables de entorno configurar y qué puertos exponer.

Características clave de ECS:

- **Orquestación completa:** gestiona despliegues, actualizaciones y escalado de contenedores.
- **Flexibilidad de infraestructura:** puede ejecutarse sobre instancias EC2 administradas por el usuario o sobre Fargate de forma *serverless*.

- **Integración con AWS:** se conecta de forma nativa con servicios como IAM (para permisos y seguridad), ELB (para balanceo de carga), CloudWatch (para métricas y logs), y ECR (Amazon Elastic Container Registry para almacenar imágenes).
- **Alta disponibilidad:** los contenedores se distribuyen automáticamente entre diferentes instancias o zonas de disponibilidad para garantizar resiliencia.

ECS resulta ideal cuando se requiere un entorno altamente integrado con AWS y una curva de aprendizaje menor en comparación con Kubernetes, ya que abstrae la complejidad de la orquestación con un enfoque simplificado.

3.2 AWS Fargate

AWS Fargate es un motor de cómputo *serverless* que permite ejecutar contenedores sin necesidad de aprovisionar o gestionar servidores virtuales. En lugar de crear un clúster de instancias EC2, Fargate se encarga de asignar automáticamente los recursos de CPU y memoria necesarios para cada tarea o servicio.

Entre sus principales ventajas, podemos encontrar:

- **Sin gestión de infraestructura:** no hay que administrar instancias EC2, ni clústeres, ni sistemas operativos.
- **Escalado automático:** ajusta la capacidad en función de la demanda.
- **Pago por uso:** se factura por los recursos solicitados (vCPU y memoria) mientras el contenedor está en ejecución.

- **Seguridad reforzada:** cada tarea corre en su propio entorno aislado, con integración nativa con IAM.

En la práctica, se suele utilizar para:

- **Tareas puntuales o batch** que requieren ejecución temporal.
- **Aplicaciones web y APIs** que necesitan alta disponibilidad y escalabilidad sin gestionar infraestructura.
- **Workloads de microservicios** que demandan despliegues rápidos, con independencia del hardware subyacente.

3.3 ECS vs Fargate: Comparación práctica

Aspecto	Amazon ECS (Elastic Container Service)	AWS Fargate
Tipo de servicio	Servicio de orquestación de contenedores que gestiona despliegues en clústeres EC2 o en Fargate	Motor de cómputo serverless para ejecutar contenedores sin gestionar servidores
Gestión de infraestructura	El usuario puede elegir: gestionar instancias EC2 propias o usar Fargate como backend	No se gestionan instancias EC2 ni clústeres; AWS asigna los recursos automáticamente
Configuración	Se definen tasks (imagen, CPU, memoria, variables de entorno, puertos)	Se definen tasks igual, pero solo se especifica CPU y memoria; Fargate hace el resto

Escalabilidad	Escala en función de las instancias EC2 disponibles o mediante Fargate	Escalado automático, sin preocuparse por servidores subyacentes
Pago	Pago por los recursos de EC2 y servicios asociados (balanceo, almacenamiento)	Pago por uso de vCPU y memoria mientras corren los contenedores
Integración con AWS	Integración nativa con IAM, ELB, CloudWatch, ECR, etc.	También integración nativa con IAM y servicios AWS, pero con entornos aislados por tarea para más seguridad
Alta disponibilidad	Distribuye contenedores entre instancias y zonas para resiliencia	Cada tarea se ejecuta en un entorno aislado, escalando y distribuyendo automáticamente
Curva de aprendizaje	Más sencilla que Kubernetes, pero implica conocer instancias EC2 si no se usa Fargate	Aún más simple porque no hay infraestructura que gestionar
Casos de uso ideales	Entornos integrados con AWS con necesidad de control de infraestructura o flexibilidad EC2/Fargate	Tareas puntuales, batch, microservicios y aplicaciones web con alta disponibilidad sin gestionar servidores

En general, **Amazon ECS sobre EC2** es más adecuado cuando se desea tener control sobre la infraestructura subyacente (tipo de instancia, almacenamiento, configuración de red, optimización de costos en cargas estables). Por otro lado,

Fargate está orientado a escenarios *serverless*, donde la prioridad es evitar la gestión de servidores, ganar agilidad en despliegues y pagar solo por el tiempo de ejecución.

Muchas arquitecturas en AWS combinan ambos: ECS como orquestador y Fargate como motor de ejecución para workloads que requieren flexibilidad o escalabilidad sin intervención manual.

4. AWS Lambda

AWS Lambda es un servicio de computación sin servidor (*serverless*) que permite ejecutar fragmentos de código en la nube sin necesidad de aprovisionar ni administrar servidores. En este modelo, la ejecución está orientada a eventos: el código se activa únicamente cuando se produce un desencadenador (*trigger*) y el usuario paga solo por el tiempo de ejecución y los recursos consumidos. Esto elimina los costos asociados a la capacidad ociosa y permite construir aplicaciones y procesos altamente escalables de forma ágil y eficiente.

Este paradigma permite desarrollar soluciones con un coste optimizado, ya que no se paga por capacidad inactiva.

Activación y desencadenadores

Las funciones Lambda pueden activarse a partir de distintos tipos de eventos generados por servicios de AWS o por aplicaciones externas. Algunos ejemplos comunes incluyen:

- Subida de archivos a un bucket de Amazon S3.
- Llegada de mensajes a una cola en Amazon SQS o a un tema en Amazon SNS.

- Cambios en tablas de Amazon DynamoDB.
- Flujos de datos en tiempo real con Amazon Kinesis.
- Ejecuciones programadas mediante Amazon EventBridge (antes CloudWatch Events).
- Solicitudes HTTP recibidas a través de Amazon API Gateway.

Cada función se asocia a un *trigger*, **el cual define cómo y cuándo se ejecuta**. Gracias a la integración nativa con el ecosistema AWS, es posible construir flujos complejos sin necesidad de desarrollar código adicional para la orquestación.

Escalabilidad automática

Uno de los aspectos más potentes de Lambda es su escalabilidad automática. El servicio ajusta de manera transparente el número de instancias de ejecución de una función en función del volumen de eventos recibidos. Esto significa que la misma aplicación puede pasar de manejar unos pocos eventos por día a procesar miles o millones de solicitudes por segundo sin cambios en la infraestructura ni intervención del usuario.

El escalado se produce de manera horizontal: si múltiples eventos llegan al mismo tiempo, Lambda lanza varias instancias de la función en paralelo para atenderlos, garantizando tiempos de respuesta óptimos incluso durante picos de demanda inesperados.

Integración y monitoreo

La integración con el ecosistema AWS es otra de sus grandes ventajas. Lambda se conecta de forma nativa con la mayoría de los servicios de la plataforma, lo que permite, por ejemplo, ejecutar funciones cada vez que un archivo es cargado en S3, responder a actualizaciones en DynamoDB, automatizar tareas

de infraestructura con CloudWatch o construir APIs completas junto con API Gateway de manera completamente *serverless*.

Además, los desarrolladores cuentan con servicios de monitoreo y observabilidad como Amazon CloudWatch Logs y AWS X-Ray, que facilitan la recolección de métricas, la trazabilidad de ejecuciones y la depuración de errores en entornos productivos.

Lenguajes y despliegue

AWS Lambda soporta una amplia variedad de lenguajes, incluyendo Python, Node.js, Java, Go, Ruby y C#. El código puede escribirse directamente en la consola de AWS, desarrollarse localmente con herramientas como AWS SAM (*Serverless Application Model*) o desplegarse mediante *pipelines* de CI/CD utilizando frameworks como Serverless Framework. Esta flexibilidad permite que equipos con distintos *stacks* tecnológicos adopten Lambda sin fricciones, integrándolo fácilmente en sus procesos de desarrollo y despliegue.

Casos de uso y limitaciones

Los casos de uso de Lambda son muy diversos. Es especialmente útil para:

- Procesamiento de datos en tiempo real.
- Automatización de flujos de trabajo y tareas administrativas.
- Integración entre sistemas mediante eventos asincrónicos.
- Creación de microservicios y APIs ligeras en combinación con API Gateway.
- Ejecución de trabajos programados recurrentes.

Entre sus principales ventajas se destacan la escalabilidad automática, el modelo de facturación por uso, la reducción de la complejidad operativa al no requerir servidores ni infraestructura gestionada, y la facilidad de despliegue e integración con otros servicios de AWS.

No obstante, es importante tener presentes algunas limitaciones: **cada invocación de Lambda tiene un tiempo máximo de ejecución de 15 minutos, la memoria está limitada a 10 GB** y no es la mejor opción para cargas de trabajo que requieren cómputo intensivo de larga duración o procesamiento continuo.

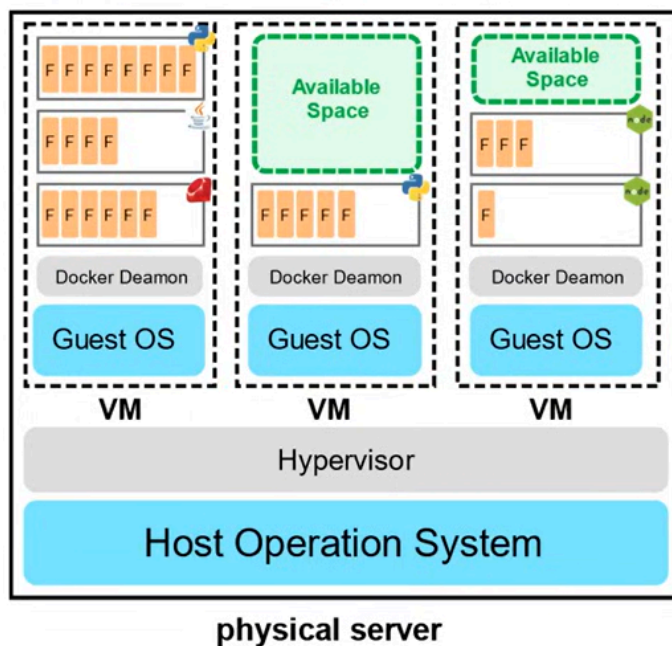


Figura 3: Comparativa visual de EC2 vs Contenedores vs Lambda Functions.

En conclusión, AWS Lambda representa la propuesta serverless de Amazon, orientada a simplificar la ejecución de aplicaciones y procesos bajo demanda. Su integración con el ecosistema AWS, su capacidad de escalar de forma automática y su modelo de pago por uso lo convierten en una de las

herramientas más poderosas y adoptadas dentro del paradigma de computación sin servidores.

5. Conclusión

En esta unidad exploramos en detalle los componentes fundamentales que forman parte de una instancia EC2 en AWS. Comprendimos cómo las instancias EC2 funcionan como servidores virtuales en la nube y se adaptan a distintos escenarios según el tipo de instancia elegido. Vimos que las Amazon Machine Images (AMI) permiten estandarizar configuraciones iniciales y simplificar el despliegue de entornos. También analizamos el rol de los volúmenes EBS, que proveen almacenamiento persistente para el sistema operativo y los datos, y cómo el CPU y la memoria varían según las necesidades de cómputo. Además, repasamos la importancia de las interfaces de red (ENI) para la conectividad dentro de una VPC y el rol de los grupos de seguridad como primera capa de protección frente a accesos no deseados.

Estos conceptos son clave porque nos permiten entender cómo se compone y asegura una instancia en la nube, logrando flexibilidad, escalabilidad y seguridad al diseñar arquitecturas en AWS.

6. Glosario de conceptos clave

- **EC2 (Elastic Compute Cloud):** Servicio de AWS que permite crear y gestionar servidores virtuales en la nube, con control total sobre el sistema operativo, CPU, memoria y almacenamiento.

- **Instancia EC2:** Servidor virtual desplegado sobre la infraestructura de AWS. Cada instancia funciona como un equipo independiente y puede escalar según la demanda.
- **Amazon Machine Image (AMI):** Plantilla que define el sistema operativo y software base para lanzar una instancia EC2. Puede ser proporcionada por AWS, creada por el usuario o adquirida en el Marketplace.
- **EBS (Elastic Block Store):** Volúmenes de almacenamiento persistente que se pueden asociar a instancias EC2. Incluyen discos para el sistema operativo y discos de datos adicionales.
- **CPU y Memoria:** Recursos de cómputo asignados a una instancia EC2, determinados por el tipo de instancia seleccionada (por ejemplo, t3.micro, m6i.large).
- **ENI (Elastic Network Interface):** Interfaz de red virtual que conecta una instancia EC2 a una VPC, permitiendo asignar IP privada y opcionalmente IP pública.
- **Grupos de Seguridad (Security Groups):** Firewalls virtuales que controlan el tráfico entrante y saliente de las instancias EC2, mediante reglas de red configurables.
- **Auto Scaling:** Servicio que ajusta automáticamente la cantidad de instancias EC2 en función de la demanda, manteniendo rendimiento y disponibilidad.
- **Elastic Load Balancing (ELB):** Servicio que distribuye el tráfico de red entre múltiples instancias EC2 para mejorar la disponibilidad y eficiencia de las aplicaciones.

- **Escalado Vertical (Scale Up):** Incremento de recursos (CPU, RAM, almacenamiento) de una única instancia para manejar mayor carga.
- **Escalado Horizontal (Scale Out):** Adición de más instancias EC2 para distribuir la carga de trabajo y aumentar la capacidad de forma paralela.
- **Serverless / Lambda:** Modelo de cómputo sin servidores, donde se ejecuta código en respuesta a eventos y se paga únicamente por el tiempo y recursos consumidos.
- **Contenedores:** Unidades que empaquetan aplicaciones junto con todas sus dependencias, permitiendo portabilidad y ejecución consistente en distintos entornos.
- **Amazon ECS (Elastic Container Service):** Servicio de orquestación de contenedores gestionado por AWS, que permite ejecutar y escalar aplicaciones en contenedores de manera eficiente.
- **AWS Fargate:** Motor serverless para contenedores que permite ejecutar aplicaciones sin necesidad de gestionar servidores o clusters subyacentes.
- **Trigger / Evento:** Condición o acción que activa la ejecución de una función serverless (por ejemplo, un mensaje en cola, un cambio en almacenamiento, una solicitud HTTP).
- **VPC (Virtual Private Cloud):** Red virtual aislada dentro de AWS, donde se despliegan instancias EC2, contenedores y otros recursos, con control completo sobre subredes, enrutamiento y seguridad.

- **High Availability (Alta Disponibilidad):** Capacidad de un sistema para mantenerse operativo y accesible aun cuando uno o varios componentes fallen.
- **Billing por Uso / Pay-as-you-go:** Modelo de cobro donde solo se paga por los recursos realmente utilizados, permitiendo optimización de costos en la nube.



Bibliografía utilizada y sugerida

- Microsoft. (2025). Fundamentos de Azure: Guía de estudio (AZ-900). Microsoft Learn. Recuperado de <https://learn.microsoft.com/es-es/credentials/certifications/resources/study-guides/az-900>
- Microsoft. (2025). Microsoft Learn: Ruta de aprendizaje de Fundamentos de Azure. Microsoft Learn. Recuperado de <https://learn.microsoft.com/es-es/training/paths/azure-fundamentals/>